

Quilt: Custom UIs for Linking Unstructured Documents to Structured Datasets

Pragya Kallanagoudar
UC Berkeley
Berkeley, USA
pkallanagoudar@berkeley.edu

Chithra Anand
UC Berkeley
Berkeley, USA
chithra.rajan@berkeley.edu

Rebecca M. M. Hicke
Cornell University
Ithaca, USA
rmh327@cornell.edu

Rolando Garcia
UC Berkeley
Berkeley, USA
rogarcia@berkeley.edu

Aditya Parameswaran
UC Berkeley
Berkeley, USA
adityagp@berkeley.edu

Eunice Jun
UCLA
Los Angeles, USA
emjun@cs.ucla.edu

Sarah E. Chasins
UC Berkeley
Berkeley, USA
schasins@eecs.berkeley.edu

ABSTRACT

Practitioners working with large document dumps struggle to identify and record connections between documents and structured datasets. For example, how should journalists building a database of police misconduct incidents connect police report documents to their database of officers? The current approach is either to (i) switch between reading entire documents and querying databases, which is a tedious manual process, or (ii) build a custom linking UI to help. These linking UIs are typically built from scratch using vanilla web programming, despite having similar structures. We introduce Quilt, a framework for generating custom UIs for linking between documents and databases. Quilt’s API lets programmers provide domain knowledge—e.g., what counts as evidence that a particular database row is relevant to a document. From this information, Quilt generates the UI automatically. We explore Quilt’s expressivity via four case studies and find it handles a diverse range of documents and databases.

CCS CONCEPTS

• **Human-centered computing** → **User interface programming**.

KEYWORDS

UI generation, API design, entity linking, document data dumps

ACM Reference Format:

Pragya Kallanagoudar, Chithra Anand, Rebecca M. M. Hicke, Rolando Garcia, Aditya Parameswaran, Eunice Jun, and Sarah E. Chasins. 2024. Quilt: Custom UIs for Linking Unstructured Documents to Structured Datasets. In *The 37th Annual ACM Symposium on User Interface Software and Technology*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST Adjunct ’24, October 13–16, 2024, Pittsburgh, PA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0718-6/24/10

<https://doi.org/10.1145/3672539.3686777>

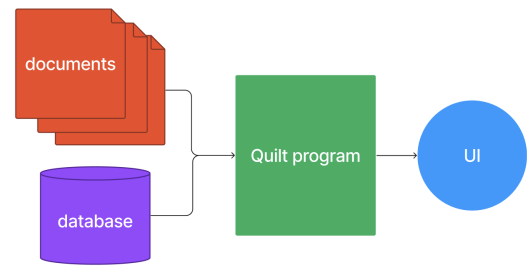


Figure 1: Quilt System Architecture. A Quilt program accepts as input (i) a set of documents, (ii) a database of structured data about entities. The Quilt program itself includes seven programmer-authored functions providing details about the domain—e.g., are there strings whose presence in a document constitute evidence that a given entity should be linked to the document? The output of a Quilt program is a custom, domain-specific UI that supports end users in linking unstructured documents with the structured information in a database.

(UIST Adjunct ’24), October 13–16, 2024, Pittsburgh, PA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3672539.3686777>

1 INTRODUCTION

In many domains, a substantial fraction of information is recorded in unstructured documents such as PDFs [8, 12]. Imposing structure on these documents is essential to put them to use for analysis, sensemaking, and search. Typically, structure is imposed by linking these documents to existing structured information in these domains [8]. For example, analysis of police incident reports to (i) find all reports relevant to a given officer, and (ii) synthesize patterns

of misconduct, is only possible after document-database linking is performed. Consider the following scenario from our collaborators at the National Association of Criminal Defense Lawyers (NACDL):

A team of public defenders wants to track police officers involved in police misconduct cases. They have a collection of court case documents from the last ten years and a database containing information about police officers, their affiliations, and dates of employment. They aim to create a new dataset that connects these documents with the officers mentioned in them.

The current manual approach of linking documents and datasets is difficult and time-consuming. Domain experts like these public defenders manually read through the entirety of each document. When they come across a name, they query the database to see if it appears in their database of officers. If so, and if the details in the document convince them that the document references the database entry in question, they update a separate table with the link between the document and the database record. This is repeated for all documents. We refer to this kind of task as a *document-database linking task*.

In contrast to tasks like entity linking (EL) [1, 14], many document-database linking tasks require a human in the loop. A mistake can be catastrophic—e.g., linking the wrong officer to a legal case surrounding misconduct in a database intended for use in the justice system. In addition to the high stakes, the difficulty of the task also means this process requires manual supervision, and cannot be simply automated away. For example, it is often not clear which person is being referenced in a case (e.g., if multiple individuals share the same name), and this requires manual inspection into individuals' work histories to confirm a link.

Other tools, such as entity recognition [10] tools, can get users part of the way. Tools like Amazon Comprehend [3], width.ai [9], and Wondershare PDFelement [18], can identify named entities in documents (Named Entity Recognition or NER). However, NER alone is insufficient because (i) NER does not help users make the links to existing structured data, and (ii) NER identifies a fixed set of entity types, such as human names or location names, but users may have structured data representing any number of data types. They have no guarantee that off-the-shelf NER systems recognize mentions of their target data type.

The document-database linking challenge is not unique—it is widespread across a diverse array of domains and data types. While bespoke web development can provide domain-specific linking interfaces, the resultant codebases are typically large and complex. The prevalence of the need for document-database linking tools, combined with the complexity of the traditional web programming style, suggests an opportunity for a framework that can scaffold document-database linking. This paper presents such a framework.

Generalizing the task structure to express custom interfaces for a diverse range of domains and datasets is non-trivial [5, 6, 11, 13, 17]. To achieve a general framework, we needed to meet a set of requirements that involved balancing the needs of two different user bases:

- (1) the **programmer**, who writes a program that uses the framework to generate an interface
- (2) the **end user**, who uses the generated interface

These design requirements are discussed further in Section 2.

This framework is closely related to APIs for generating user interfaces. For example, Mavo extends HTML to enable lightweight development of interactive web applications that can also update data [15, 16]. In a less general-purpose context, Wikxhibit is a Javascript library for pulling data from WikiData and other connected APIs and displaying it in an interactive webpage [2]. Like Wikxhibit, our framework offers an API for authoring a specialized kind of web application.

We describe Quilt (*Quick User Interfaces for Linking Tool*), a framework for developing human-in-the-loop document-database linking interfaces. Tools generated with the Quilt API let users (i) find mentions of entities and entity-related information in unstructured documents (e.g., PDF files) and (ii) link the documents to records in existing databases. Quilt-generated tools pre-process both the structured and unstructured data sources to identify and rank potential links. These possible links are then shown alongside the document so the end user can make informed decisions about which records from the database are relevant to the document.

The programmer can control what information is used for suggesting and ranking candidate links (Is a police department's name evidence that a given police officer is relevant to a document?), what database information the Quilt API displays for end users (Should the interface show an officer's employment history?), and what information the Quilt interface highlights in the document (Should mentions of an officer's middle name be highlighted?), among other details of the final output UI.

2 DESIGN PROCESS AND GOALS

Quilt is the result of an ongoing collaboration with domain experts from the National Association for Criminal Defense Lawyers (NACDL). Our collaborators requested an interface to help domain experts identify police officers in documents for civil rights complaints. Over the course of approximately two years, we met weekly with two stakeholders from NACDL and one journalist stakeholder from KQED, a public radio station. In those meetings, we discussed their needs, identified tool requirements, iterated on designs, conducted critique sessions, demonstrated tool prototypes, and received continual feedback.

Our co-design process helped us develop the following design goals for supporting both *programmers* and *end users*:

- **DG1 - Developer flexibility.** Programmers should be able to control: what the UI surfaces as relevant entity properties and attributes, how entities are grouped, and how entities are ranked within groups. This gives programmers the flexibility to support diverse end-user needs and application domains.
- **DG2 - End-user ground truth control.** End users should have ultimate control over which entities are matched to which documents. This is especially important when the database includes many very similar entities, which could lead to mistakes in an automated system.
- **DG3 - Explainable inferences.** In the event that end users must pick among multiple candidate entity matches, the Quilt-generated interface should support the user in making these decisions. End users should be able to inspect how

Quilt determined which entities are relevant to the document, grouped entities, and ranked them within groups. This may help end users audit the process and make informed decisions when overriding Quilt’s suggestions.

3 SYSTEM ARCHITECTURE

The pipeline to generate the matching interface begins when the programmer calls run on a Quilt instance, passing a list of documents and the database (as a pandas dataframe). Throughout the pipeline, the Quilt runtime calls out to a set of seven programmer-provided functions; programmers can use these Quilt API functions to encode the key domain knowledge relevant to making connections between documents and databases. See Figure 2 for a representation of the inputs and outputs after the use of each Quilt API function. The run function executes the following steps for each document:

Document Processing. At the start of the pipeline, every document is processed to produce: (a) its full text as a string (using OCR if necessary), (b) an image of each page of the document, and (c) the coordinates of each token on each image. The subsequent steps use these processing outputs for each document and consider each document separately.

Entity Matching. Next, Quilt uses the programmer’s implementation of Quilt API function `entity_relevant_to_doc` to identify all entities from the database that are relevant to the document. Quilt groups the relevant entities based on the programmer-provided `group_entities` function.

Entity Ranking & Highlighting. Quilt sorts the entities in each entity group by using the programmer-provided `rank_relevance_of_entities_to_doc` function. Quilt calls the programmer-provided `strings_to_highlight` function on each entity in an entity group to determine strings that should be highlighted while the entity group is selected. All the relevant strings and coordinates for each entity in a group are aggregated so that they can be highlighted together in the UI.

UI Creation. Quilt uses the display functions to determine the labels for entity groups and entities in the side panel of the UI, as well as to generate the additional supporting text to display for each entity. In its final form, the side panel includes an accordion element for each entity group, then a nested accordion element for each entity within the entity group. When a particular entity group is selected in the side panel, the highlights for the relevant strings are activated. The output UI also implements all of the requisite interactivity for marking particular entities as relevant, possibly relevant, and not relevant—and for generating the associated output data based on those interactions. The final web application includes a navigation page that displays a gallery of all of the input documents. Clicking on any of the documents navigates the user to the document-specific view described above.

4 CASE STUDIES

We explored the expressivity of the Quilt API through three case studies. In the first case study, Quilt was used to create an interface for a user to identify medical acronyms from a database of medical

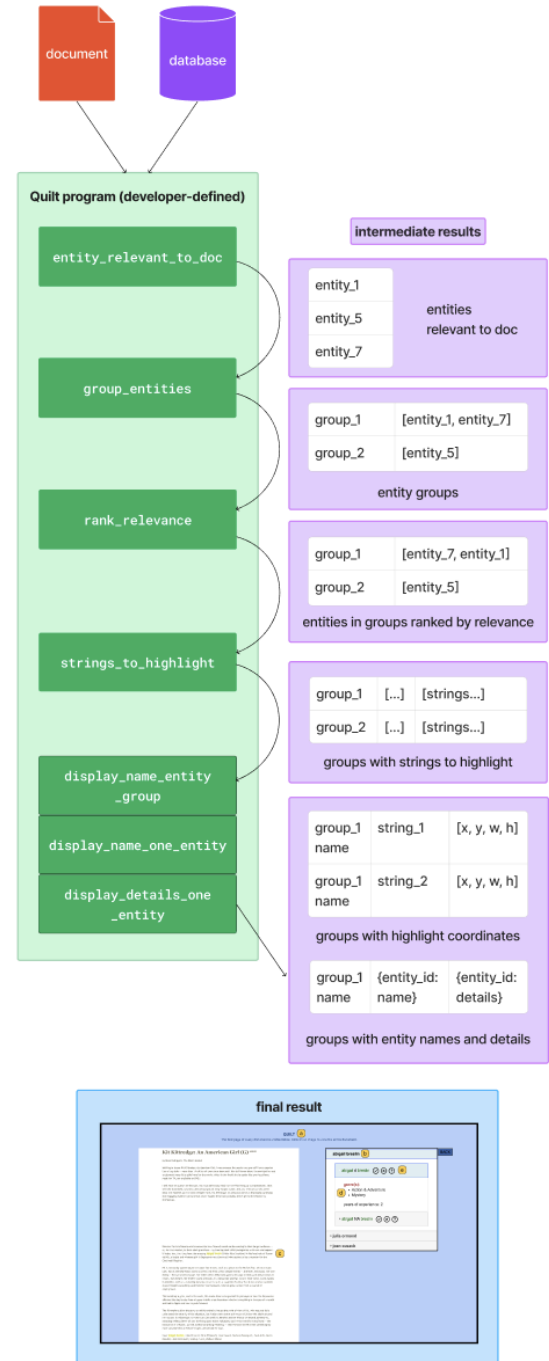


Figure 2: System architecture.

Each document in the corpus goes through four stages of the Quilt pipeline (section 3): document processing, entity matching, entity ranking, and UI creation.

terms in clinical texts, which is a disambiguation problem in the biomedical sciences [7]. In the second case study, Quilt was used to create an interface for a user to identify organizations in patents,

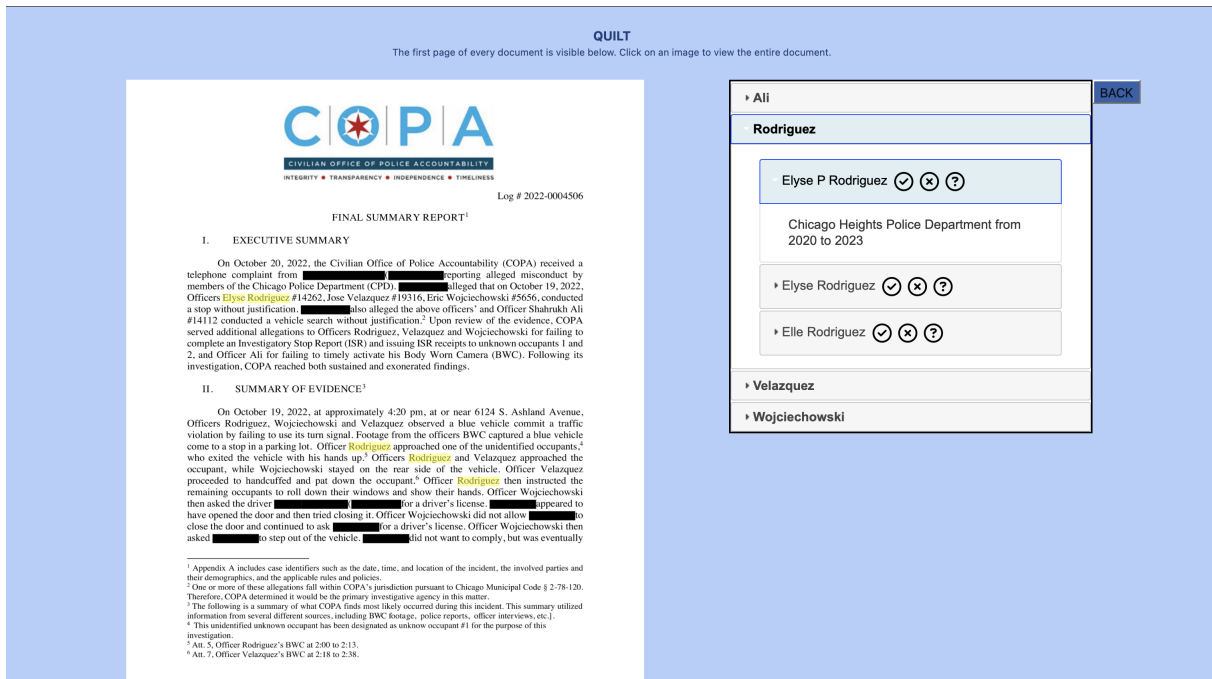


Figure 3: Quilt UI for Identifying Police Officers in Civil Rights Complaints

This UI is generated from an example program that a developer might write to create a Quilt interface for the use case of identifying police officers in civil rights case reports. The right-hand side panel groups officers according to their last name. The relevant strings for the selected officer, “Officer Rodriguez“, are highlighted in the document on the left-hand side.

which is motivated by a related line of research in economics [4]. In the third case study, Quilt was used for the original purpose for which it was developed— to create an interface to identify police officers in civil rights complaint documents. In this paper, we have only included the interface for the third case study (Figure 3) for brevity.

REFERENCES

- [1] [n.d.]. NLP-progress: Entity Linking. https://nlpprogress.com/english/entity_linking.html. Accessed: 2024-03-22.
- [2] Tarfah Alrashed, Lea Verou, and David Karger. 2022. Wikxhibit: Using HTML and Wikidata to Author Applications that Link Data Across the Web. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*. 1–15.
- [3] Amazon. 2024. Amazon Comprehend. <https://aws.amazon.com/comprehend/>
- [4] Olivier Binette, Sarvo Madhavan, Jack Butler, Beth Anne Card, Emily Melluso, and Christina Jones. 2023. PatentsView-Evaluation: Evaluation Datasets and Tools to Advance Research on Inventor Name Disambiguation. *arXiv preprint arXiv:2301.03591* (2023).
- [5] Krzysztof Gajos and Daniel S Weld. 2004. SUPPLE: automatically generating user interfaces. In *Proceedings of the 9th international conference on Intelligent user interfaces*. 93–100.
- [6] Krzysztof Z Gajos, Jacob O Wobbrock, and Daniel S Weld. 2007. Automatically generating user interfaces adapted to users’ motor and vision capabilities. In *Proceedings of the 20th annual ACM symposium on User interface software and technology*. 231–240.
- [7] Grossman R.H. Mitchell E.G Grossman Liu, L. 2021. A deep database of medical abbreviations and acronyms for natural language processing. *Sci Data* 8, 149 (2021).
- [8] James Hendler. 2014. Data integration for heterogenous datasets. *Big data* 2, 4 (2014), 205–215.
- [9] Width LLC. 2024. Width.ai. <https://www.width.ai/>
- [10] David Nadeau and Satoshi Sekine. 2007. A survey of named entity recognition and classification. *Linguistic Investigations* 30, 1 (2007), 3–26.
- [11] Jeffrey Nichols, Brandon Rothrock, Duen Horng Chau, and Brad A Myers. 2006. Huddle: automatically generating interfaces for systems of multiple connected appliances. In *Proceedings of the 19th annual ACM symposium on User interface software and technology*. 279–288.
- [12] David W Nickerson and Todd Rogers. 2014. Political campaigns and big data. *Journal of Economic Perspectives* 28, 2 (2014), 51–74.
- [13] Charles Rich, Candy Sidner, Neal Lesh, Andrew Garland, Shane Booth, and Markus Chimani. 2005. DiamondHelp: A Graphical User Interface Framework for Human-Computer Collaboration. In *IEEE International Conference on Distributed Computing Systems Workshops*. 514–519.
- [14] Wei Shen, Jianyong Wang, and Jiawei Han. 2014. Entity linking with a knowledge base: Issues, techniques, and solutions. *IEEE Transactions on Knowledge and Data Engineering* 27, 2 (2014), 443–460.
- [15] Lea Verou, Tarfah Alrashed, and David Karger. 2018. Extending a reactive expression language with data update actions for end-user application authoring. In *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology*. 379–387.
- [16] Lea Verou, Amy X Zhang, and David R Karger. 2016. Mavo: creating interactive data-driven web applications by authoring HTML. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*. 483–496.
- [17] Charles Wiecha, William Bennett, Stephen Boies, John Gould, and Sharon Greene. 1990. ITS: A tool for rapidly developing interactive applications. *ACM Transactions on Information Systems (TOIS)* 8, 3 (1990), 204–236.
- [18] Wondershare. 2024. Wondershare PDFelement. <https://pdf.wondershare.com/>